

## LISTA DE EXERCÍCIOS 2.8

1. Explique a diferença entre escalonamento preemptivo e não-preemptivo.

**Resposta:** Escalonamento preemptivo permite que um processo seja interrompido no meio de sua execução, tomando a CPU e alocando-a a outro processo. Escalonamento não-preemptivo assegura que um processo libere o controle da CPU somente quando ele termina seu ciclo atual de execução em CPU (CPU burst).

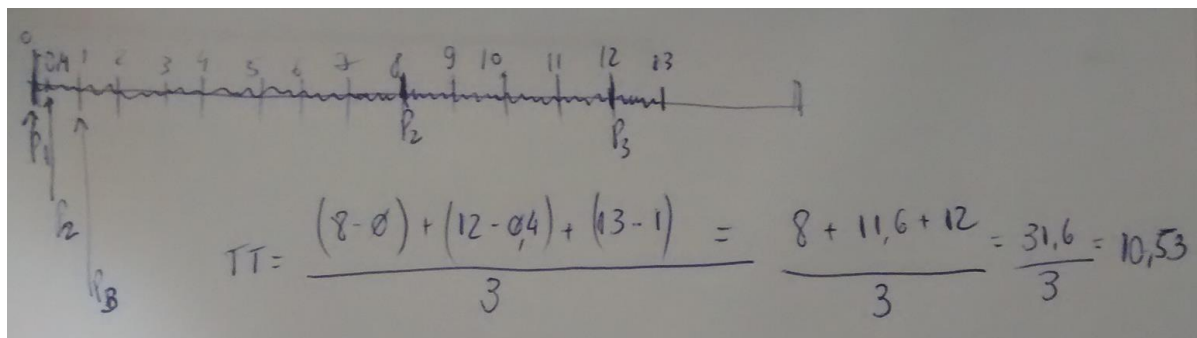
2. Suponha que os seguintes processos cheguem para execução nos tempos indicados. Cada processo irá executar pela quantidade de tempo mostrada. Para responder as questões, use **escalonamento não-preemptivo**, e baseie todas as suas decisões na informação que você possui no momento que a decisão deve ser tomada.

| Processo       | Tempo de Chegada | Tempo de CPU |
|----------------|------------------|--------------|
| P <sub>1</sub> | 0,0              | 8            |
| P <sub>2</sub> | 0,4              | 4            |
| P <sub>3</sub> | 1,0              | 1            |

Lembre-se que o *turnaround time* é calculado pelo tempo de término menos o tempo de chegada do processo, dessa forma você terá que subtrair os tempos de chegada para computar os *turnaround times*. No caso do FCFS, o valor calculado será 11 se você esquecer de subtrair o tempo de chegada.

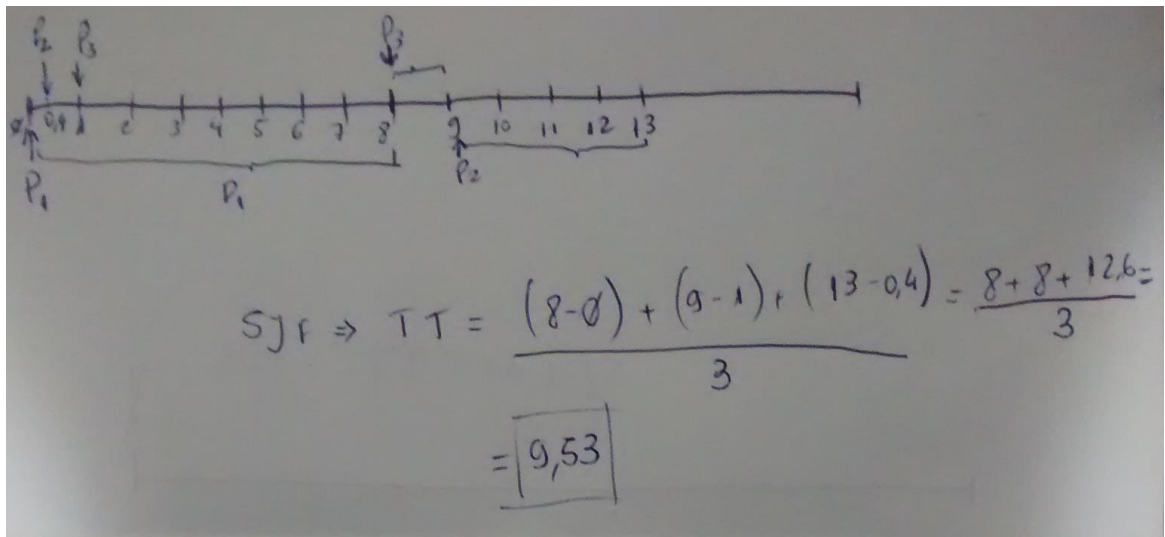
- a) Qual é o *turnaround time* médio para estes processos com o algoritmo de escalonamento FCFS.

**Resposta:** 10,53



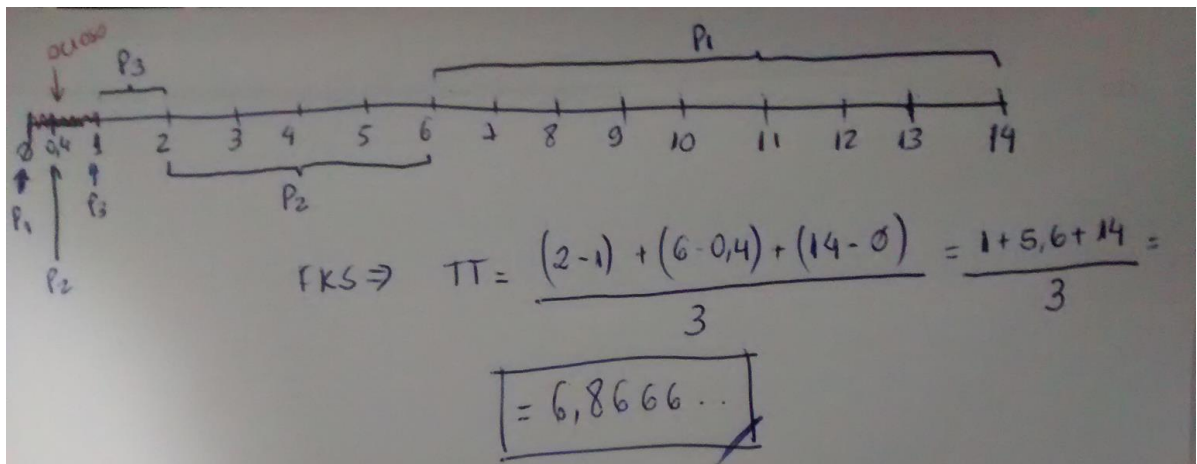
- b) Qual é o *turnaround time* médio para estes processos com o algoritmo de escalonamento SJF.

**Resposta:** 9,53



- c) Supõe-se que o algoritmo SJF deve melhorar o desempenho, mas percebe-se que foi escolhido executar o processo P<sub>1</sub> no tempo 0, pois não se sabia que dois processos com tempos de execução menores chegariam em breve. Compute qual será o *turnaround time* médio se a CPU fica ociosa pela primeira unidade de tempo e então o escalonamento SJF é usado. Lembre que os processos P<sub>1</sub> e P<sub>2</sub> estão esperando durante este tempo ocioso, de modo que seus tempos de espera podem aumentar. Este algoritmo poderia ser conhecido como escalonamento de conhecimento futuro (*future-knowledge scheduling*).

**Resposta:** 6,86



3. Qual é a vantagem em se ter diferentes tamanhos de quantum em diferentes níveis de um sistema de filas multinível?

**Resposta:** Processos que precisam executar com mais frequência, como por exemplo, no caso de processos interativos (ex. editores de texto), podem estar em uma fila com um pequeno tamanho de quantum. Processos que não precisem executar com tanta frequência, podem ser colocados em uma fila com um tamanho de quantum maior, requerendo uma quantidade menor de chaveamentos de contexto para completar o processamento, e assim, fazer uso mais eficiente do computador.

4. Muitos algoritmos de escalonamento de CPU são parametrizados. Por exemplo, o algoritmo RR requer um parâmetro para indicar o intervalo de tempo (*time slice*). Filas multinível por realimentação (*multilevel feedback queues*) requerem parâmetros para definir o número de filas, os algoritmos de escalonamento para cada fila, o critério usado para mover processos entre filas e assim por diante. Estes algoritmos são em verdade conjuntos de algoritmos (ex. o conjunto de algoritmos RR para todos os intervalos de tempo e assim por diante). Um

conjunto de algoritmos pode incluir outro (ex. FCFS é um RR com quantum infinito). Qual é a relação mantida entre os seguintes pares de algoritmos (se é que existe alguma):

a) Prioridade e SJF

**Resposta:** A menor tarefa (job) possui a maior prioridade.

b) Fila multinível com realimentação e FCFS

**Resposta:** O menor nível de fila multinível, em geral, é o FCFS (vide questão 3).

c) Prioridade e FCFS

**Resposta:** FCFS dá a maior prioridade para a tarefa (job) que é mais antigo (i.e. chegou a mais tempo).

d) RR e SJF

**Resposta:** Não possuem relação.

5. Suponha que um algoritmo de escalonamento (no nível de escalonamento de CPU de curto prazo) favoreça àqueles processos que usaram a menor quantidade de tempo de processamento no passado recente. Por que este algoritmo irá favorecer programas voltados a I/O (*I/O-bound*) e ainda, de forma não permanente, causar *starvation* de programas voltados a CPU (*CPU-bound*)?

**Resposta:** Este algoritmo irá favorecer programas voltados a I/O (*I/O-bound*) devido ao tempo de CPU (*CPU burst*) relativamente curto usado por eles; entretanto, os programas voltados a CPU (*CPU-bound*) não irão sofrer *starvation* (de forma permanente) pois os programas voltados a I/O, em algum momento, irão liberar a CPU com alguma frequência para realizar I/O. **O problema** surge caso exista no sistema uma grande quantidade de processos que realizam I/O. Se somado a quantidade de processos, estes também encerram suas atividades de I/O de forma mais ou menos intercalada, os processos voltados a I/O (que já foram processados) retornem a executar na frente dos processos voltados a CPU devido a sua prioridade maior. Neste caso, a prioridade ficará para todos esses processos que realizam I/O e, só após o processamento do ciclo de CPU de cada um destes (e talvez continuação de processamento de alguns), será cedida a vez para processos voltados a CPU. Percebe-se que os processos voltados a CPU basicamente terão como *slot* de tempo de execução apenas o tempo em que os processos voltados a I/O estiverem executando I/O (cogitando-se que não haja o intercalamento entre os processo que executam I/O – isso pode de fato levar a *starvation*).

6. Diferencie algoritmos PCS e SCS.

**Resposta:** escalonamento PCS (escopo de contenção de processo) é realizado localmente ao processo. É assim que a biblioteca de *thread* escala *threads* em LWPs disponíveis. O escalonamento SCS (escopo de contenção de sistema) ocorre quando o SO escala *threads* de kernel. Em sistemas usando a política muitos-para-um e muitos-para-muitos, os dois modelos de escalonamento são fundamentalmente diferentes. Em sistemas usando um-para-um, PCS e SCS são idênticos. ---  
**DISCUSSÃO:** por que são fundamentalmente diferentes? Resposta: por que o PCS refere-se apenas as *threads* de um processo e o SCS refere-se a todas as *threads* do sistema. Assim sendo, no modelo muitos-para-um ou muitos-para-muitos o escalonamento PCS será referente apenas a um processo enquanto o SCS refere-se a todas as *threads* do sistema.

7. Assuma que um SO mapeie *threads* de espaço de usuário para o kernel usando o modelo muitos-para-muitos e que o mapeamento é realizado por meio de LWPs. Além disso, o sistema permite que desenvolvedores de programa criem *threads* de tempo real. É necessário atrelar uma *thread* de tempo real a um LWP?

**Resposta:** Sim. De outra forma, uma *thread* de usuário pode ter que competir por um LWP disponível antes de ser realmente escalonada. Por meio do atrelamento de uma *thread* de usuário a um LWP, não existe latência enquanto se espera por um LWP disponível. E nesse caso, observe que a *thread* de usuário de tempo real pode ser escalonada imediatamente.

8. O escalonador tradicional do UNIX reforça uma relação inversa entre números de prioridades e prioridades: quanto maior o número, menor a prioridade. O escalonador recalcula prioridades de processo uma vez por segundo usando a seguinte função:

$$\text{Prioridade} = (\text{uso recente de CPU} / 2) + \text{base}$$

onde base = 60 e o uso de CPU recente refere-se a um valor indicando quão frequentemente um processo usou a CPU desde o momento em que as prioridades foram recalculadas por último. Assuma que o uso recente de CPU para os processos P1 é 40, para o processo P2 é 18 e para o processo P3 é 10. Quais serão as novas prioridades para estes três processos quando as prioridades são recalculadas? Baseado nesta informação, o escalonador tradicional do UNIX aumenta ou diminui a prioridade relativa de processos voltados a CPU (CPU-bound)?

**Resposta:** De acordo com a figura, a prioridade diminui para processos mais recentes. Dessa forma, o escalonador diminui a prioridade relativa de processos voltados a CPU, tendo em vista que estes sempre terão os acessos mais recentes à CPU.

$$\text{Prioridade} = (\text{uso recente de CPU} / 2) + \text{base}$$

base = 60

| Proc           | Uso recente de CPU | Prior.             |
|----------------|--------------------|--------------------|
| P <sub>1</sub> | 40                 | $(40/2) + 60 = 80$ |
| P <sub>2</sub> | 18                 | $(18/2) + 60 = 69$ |
| P <sub>3</sub> | 10                 | $(10/2) + 60 = 65$ |

↑  
Resposta 1

Handwritten annotations: "mais recente" points to P<sub>1</sub>; "mais antigo" points to P<sub>3</sub>; "menor prioridade" points to the priority value 80; "maior prioridade" points to the priority value 65.